

EXPERIMENT – 5

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Inter-Process Communication using Anonymous Pipes

Aim

To implement an anonymous pipe for producer-consumer communication and implement the command pipeline `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()`.

Theory

- A **pipe** provides a communication channel between processes.
- `pipe()` creates the pipe and provides file descriptors for reading and writing.
- `fork()` creates child processes.
- `dup2()` redirects standard input or output to the pipe.
- `exec()` executes another program.
- In the command `ls -l | grep ".c"`, the output of `ls -l` is connected to the input of `grep`.

Algorithm

1. Start the program.
2. Create an anonymous pipe using `pipe()`.
3. Create the first child process using `fork()`.
4. Redirect the first child's standard output to the pipe using `dup2()`.
5. Execute `ls -l` using `execlp()`.
6. Create the second child process using `fork()`.
7. Redirect the second child's standard input from the pipe using `dup2()`.
8. Execute `grep ".c"` using `execlp()`.
9. Close the unused pipe file descriptors.
10. Use `waitpid()` to wait for both child processes.
11. Terminate the program.

Program Code

```
k_anusri@DESKTOP-TM20BAQ:~$  
k_anusri@DESKTOP-TM20BAQ:~$ cat > exp5_pipe.c <<'EOF'  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
  
int main()  
{  
    int pipefd[2];  
    pid_t p1, p2;  
  
    if (pipe(pipefd) == -1)  
    {  
        perror("pipe");  
        return 1;  
    }  
  
    p1 = fork();  
  
    if (p1 < 0)  
    {  
        perror("fork");  
        return 1;  
    }  
  
    if (p1 == 0)  
    {  
        close(pipefd[0]);  
  
        dup2(pipefd[1], STDOUT_FILENO);  
        close(pipefd[1]);  
  
        execlp("ls", "ls", "-l", NULL);  
  
        perror("execlp");  
        exit(1);  
    }  
  
    p2 = fork();  
  
    if (p2 < 0)  
    {  
        perror("fork");  
        return 1;  
    }  
  
    if (p2 == 0)  
    {  
        close(pipefd[1]);  
  
        dup2(pipefd[0], STDIN_FILENO);  
        close(pipefd[0]);  
  
        execlp("grep", "grep", ".c", NULL);  
  
        perror("execlp");  
        exit(1);  
    }  
  
    close(pipefd[0]);  
    close(pipefd[1]);  
  
    waitpid(p1, NULL, 0);  
    waitpid(p2, NULL, 0);  
  
    return 0;  
}
```

Execution and Screenshots

Screenshot 1 — Compilation

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc exp5_pipe.c -o exp5_pipe
```

Screenshot 2 — Program Execution

```
k_anusri@DESKTOP-TM20BAQ:~$ ./exp5_pipe
-rw----- 1 k_anusri k_anusri 720 Jul 30 05:58 command_exec.c
-rw----- 1 k_anusri k_anusri 0 Jul 30 05:34 command_exec.c.save.1
-rwxr-xr-x 1 k_anusri k_anusri 16432 Sep 17 05:49 exp1_process
-rw-r--r-- 1 k_anusri k_anusri 854 Sep 17 05:48 exp1_process.c
-rwxr-xr-x 1 k_anusri k_anusri 16224 Sep 18 04:04 exp2_filecopy
-rw-r--r-- 1 k_anusri k_anusri 671 Sep 18 04:04 exp2_filecopy.c
-rwxr-xr-x 1 k_anusri k_anusri 16264 Sep 18 04:53 exp3_process_states
-rw-r--r-- 1 k_anusri k_anusri 748 Sep 18 04:53 exp3_process_states.c
-rwxr-xr-x 1 k_anusri k_anusri 16312 Sep 18 05:18 exp4_multiple_children
-rw-r--r-- 1 k_anusri k_anusri 981 Sep 18 05:17 exp4_multiple_children.c
-rw-r--r-- 1 k_anusri k_anusri 927 Sep 18 05:28 exp5_pipe.c
-rw-r--r-- 1 k_anusri k_anusri 1064 Aug 4 12:22 experiment2.c
-rw-r--r-- 1 k_anusri k_anusri 890 Jul 30 06:00 os_command.c
-rwxr-xr-x 1 k_anusri k_anusri 16432 Jul 30 06:04 process
-rw-r--r-- 1 k_anusri k_anusri 787 Jul 30 05:47 process.c
```

Result

The anonymous pipe was successfully implemented, and the `ls -l | grep ".c"` pipeline was executed using `fork()`, `pipe()`, `dup2()`, and `exec()`.